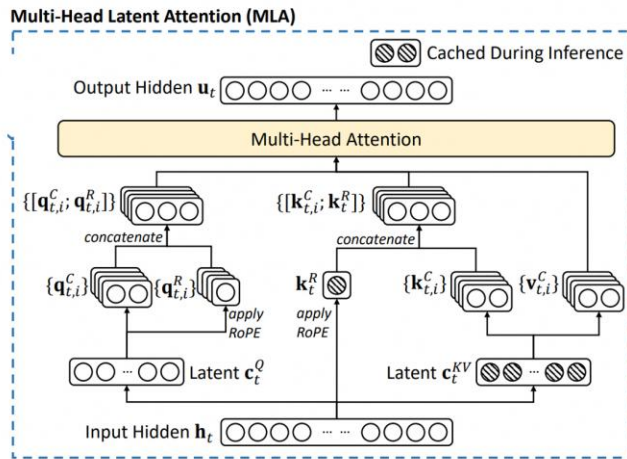


1 MLA 融合算子优化手册

1.1 MLA 介绍

1.1.1 背景

Multi-Head Latent Attention (MLA) 由 DeepSeek 团队提出并应用在 DeepSeek V3/R1 等模型，旨在通过低秩压缩技术大幅减少 KV 缓存的内存占用，同时保持甚至提升模型性能。MLA 的核心思想是将键和值投影到一个低维潜在空间，然后在这个空间中进行注意力计算，从而显著压缩 KV 缓存的大小。具体来说，MLA 通过引入低秩投影矩阵，将高维的键和值向量压缩到一个低维潜在空间，然后在这个空间中进行注意力计算。这样，在推理过程中，只需要缓存低维的潜在向量，而不是原始的高维键值对，从而大幅减少内存消耗。



低秩压缩

$$\begin{aligned}
 \mathbf{c}_t^{KV} &= W^{DKV} \mathbf{h}_t, \\
 [\mathbf{k}_{t,1}^C; \mathbf{k}_{t,2}^C; \dots; \mathbf{k}_{t,n_h}^C] &= \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, \\
 \mathbf{k}_t^R &= \text{RoPE}(W^{KR} \mathbf{h}_t), \\
 \mathbf{k}_{t,i} &= [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R], \\
 [\mathbf{v}_{t,1}^C; \mathbf{v}_{t,2}^C; \dots; \mathbf{v}_{t,n_h}^C] &= \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, \\
 \mathbf{c}_t^Q &= W^{DQ} \mathbf{h}_t, \\
 [\mathbf{q}_{t,1}^C; \mathbf{q}_{t,2}^C; \dots; \mathbf{q}_{t,n_h}^C] &= \mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q, \\
 [\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \dots; \mathbf{q}_{t,n_h}^R] &= \mathbf{q}_t^R = \text{RoPE}(W^{QR} \mathbf{c}_t^Q), \\
 \mathbf{q}_{t,i} &= [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \\
 \mathbf{o}_{t,i} &= \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C, \\
 \mathbf{u}_t &= W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}],
 \end{aligned}$$

1.1.2 定义

对于 FlashMLA 部分的优化，我们主要考虑 MLA 低秩压缩后的 Attention 计算。具体地，对与 batch 内每个 request：

输入为 $Q \in \mathbb{R}^{s_q \times h_q \times d}$, $K \in \mathbb{R}^{s_{kv} \times d}$, $V \in \mathbb{R}^{s_{kv} \times d_v}$ ，数据类型均为 bf16， s_q 由 MTP 决定， s_{kv} 为上下文 token 数量。值得注意的是，在 MLA 中 KV 共享每个 head 内前 d_v 个元素。MLA 计算公式为：

$$S = Q \cdot K^T$$

$$P = \text{softmax}(S)$$

$$O = P \cdot V$$

其中 $S \in \mathbb{R}^{s_q \times h_q \times s_{kv}}$, $P \in \mathbb{R}^{s_q \times h_q \times s_{kv}}$, $O \in \mathbb{R}^{s_q \times h_q \times d_v}$ 。特别地, S 的数据类型为 fp32, P, O 的数据类型为 bf16。

1.1.3 计算访存量化分析

根据计算公式, 可以得到 MLA 对应的理论计算量和访存量:

$$\text{计算量} = (s_q \cdot h_q \cdot s_{kv} \cdot d \cdot 2 + s_q \cdot h_q \cdot s_{kv} \cdot d_v \cdot 2) \text{ FLOPs},$$

$$\text{访存量} = (s_q \cdot h_q \cdot d + s_{kv} \cdot d + s_q \cdot h_q \cdot d_v) \text{ Bytes}.$$

在一般的长序列场景下, 可以认为 $s_{kv} \gg s_q \cdot h_q$, 因此可以估算得:

$$\text{计算访存比} = \frac{s_q \cdot h_q \cdot (d + d_v)}{d}.$$

特别地, 在 DeepSeek V3 MLA 中, $d = 576$, $d_v = 512$ 。因此, 实际计算访存比和 $s_q \cdot h_q$ 的值成正比, 下面记 $m_q = s_q \cdot h_q$ 。根据鲲鹏芯片理论算力和内存带宽, 可以推算得到当 $m_q = s_q \cdot h_q \approx 33$ 时达到计算瓶颈和访存瓶颈的临界值。

1.1.4 计算访存比与 TP 方案

在鲲鹏芯片 DeepSeek Decode 分布式推理部署方案中, 会对各进程的 MLA 计算按 TP 进行切分。记 TP 数为 N_{TP} , request 数量为 N_{req} , 特别地, DeepSeek V3 初始 head 数量为 128。TP 切分后多 request 的 Q 的形状为 $(N_{req}, s_q, \frac{128}{N_{TP}}, d_v)$, 计算访存比 $\frac{128s_q \cdot (d + d_v)}{d \cdot N_{TP}}$ 与 N_{TP} 成反比。

为了避免极端的访存瓶颈场景, 对每组 TP 进程集合的 Q 进行 AlltoAll 集合通信:

$$(N_{req}, s_q, \frac{128}{N_{TP}}, d_v) \rightarrow (\frac{N_{req}}{k}, s_q, \frac{128k}{N_{TP}}, d_v),$$

从而通过 AlltoAll 系数 k 控制计算访存比为 $\frac{128s_q \cdot k \cdot (d+d_v)}{d \cdot N_{TP}}$ 。

1.2 MLA Kernel 设计

结合 SME 指令特点，我们设计了将 m_q 按 $B_q = 32, 64$ 进行分块的两种底层 Kernel。其中 $B_q = 32$ 的 Kernel 主要针对短序列（在这里认为 $s_{kv} \leq 1K$ ）或者访存瓶颈的场景，最大程度利用内存带宽。 $B_q = 64$ 的 Kernel 则针对长序列或者计算瓶颈的场景，隐藏访存时延，最大化算力利用率。

对于 FlashMLA 端到端计算，我们会根据计算规模选择最优的底层 kernel 实现和对应的多线程负载均衡策略，实现在不同场景达成最优性能。

1.2.1 MMA 定义

我们基于 SME MOPA 指令实现 Matrix Multiply-Accumulate (MMA) 语义，进而实现矩阵乘。由于 MOPA 指令对数据排布有特定要求，为了在 MMA 的过程中实现连续访存以保证计算效率，在 MMA 之前需要对数据进行 Pack。即对于 1 次矩阵乘 $C = A \cdot B$ ，分为 3 步操作：

$$\begin{aligned}\tilde{A} &= \text{Pack}(A), \\ \tilde{B} &= \text{Pack}(B), \\ C &= \text{MMA}(\tilde{A}, \tilde{B}).\end{aligned}$$

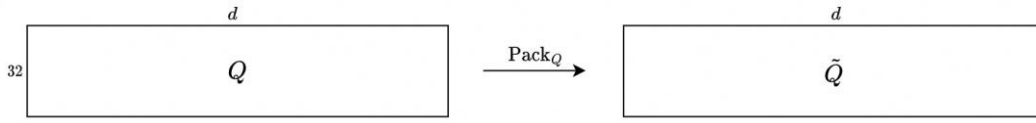
接下来将以 $B_q = 32$ 的 Kernel 为例介绍具体的数据排布和流水设计。

1.2.2 操作定义与数据排布

对于 $B_q = 32$ 的 Kernel，输入为 $Q \in \mathbb{R}^{32 \times d}$ ， $KV \in \mathbb{R}^{s_{kv} \times d}$ ，数据类型均为 bf16，初始状态下数据均在内存。

首先，我们对 Q 进行 Pack 操作，定义如下：

$$\text{Pack}_Q: \quad \tilde{Q} = \text{Pack}(Q).$$



Pack_Q 利用 SME 指令的相关特性在 ZA Tile 上进行了转置操作，因此数据流为从内存到 ZA 寄存器再到 Cache，并且我们期望在后续计算中 $\tilde{Q}$ 能够常驻在 L2 Cache，从而避免重复从内存中读取 $\tilde{Q}$ 。

接下来，我们进行 MLA 分块计算。将 KV 按 $B_{kv} = 32$ 分块，第 i 块为 KV_i ，定义分块 Pack, MMA, Softmax 操作如下：

$$\text{Pack}_{K_i}: \tilde{K}_i = \text{Pack}(KV_i),$$

$$\text{MMA}_{S_i}: S_i = \text{MMA}(\tilde{Q}, \tilde{K}_i),$$

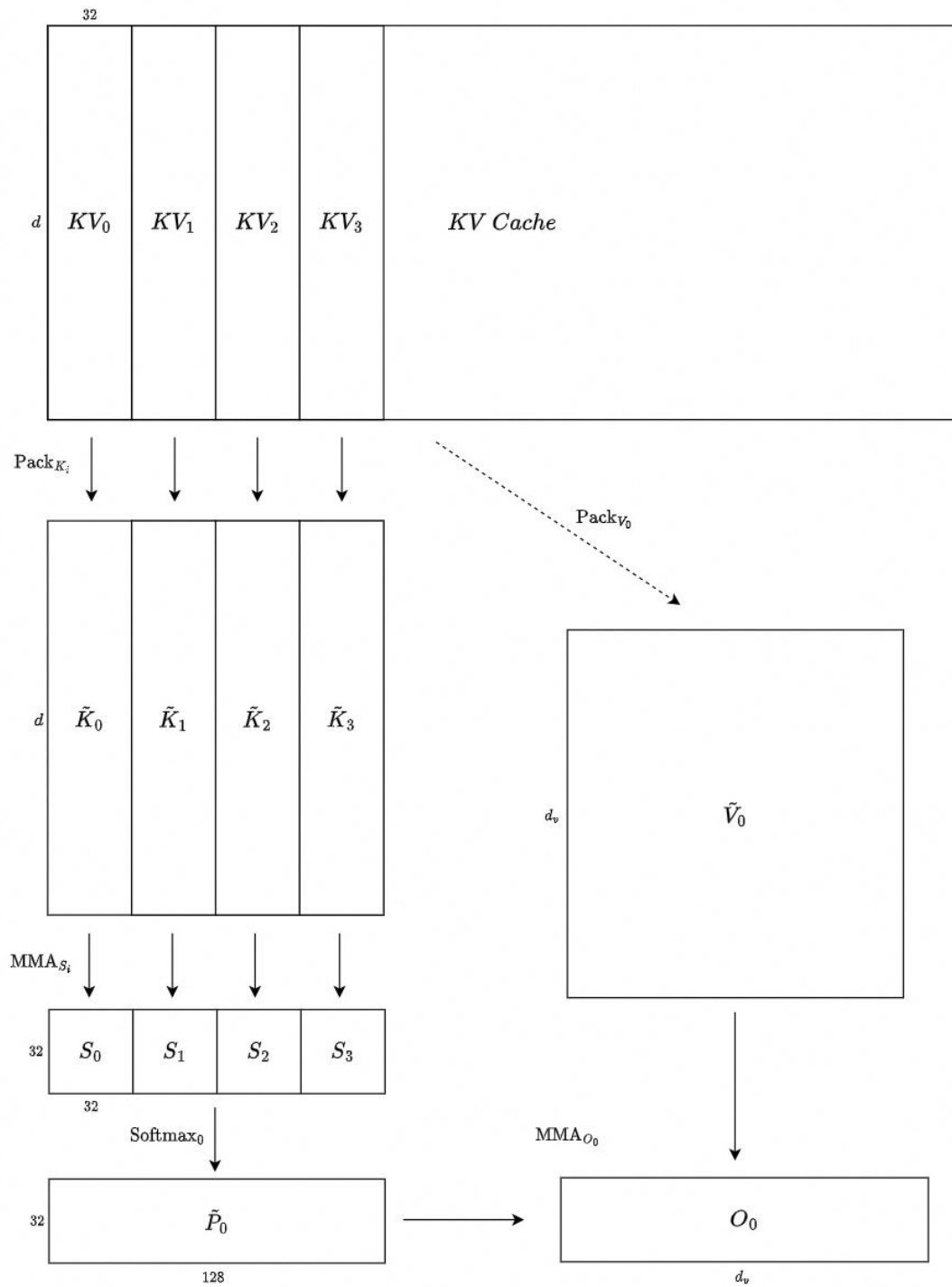
$$\text{Softmax}_i: \tilde{P}_i = \text{softmax}(\{S_{4i}, S_{4i+1}, \dots, S_{4i+3}\}),$$

$$\text{Pack}_{V_i}: \tilde{V}_i = \text{Pack}(\{KV_{4i}, KV_{4i+1}, \dots, KV_{4i+3}\}),$$

$$\text{MMA}_{O_i}: O_i = \begin{cases} \text{MMA}(\tilde{P}_i, \tilde{V}_i), & i = 0 \\ \text{scale}_i \cdot O_{i-1} + \text{MMA}(\tilde{P}_i, \tilde{V}_i), & i > 0 \end{cases}$$

我们定义关于 $KV_{4i}, KV_{4i+1}, \dots, KV_{4i+3}$ 的计算为 Phase i （第 i 轮 MLA 分块计算）。

以 Phase 0 为例，数据排布如下：



由于 $Pack_{V_i}$ 不需要用到 ZA Tile 做转置操作，可以在 MMA_{O_i} 的过程中在 SVE 寄存器中通过 SVE ZIP 指令实现，实现融合。

1.2.3 异步搬运

上面提到我们期望在分块计算中 $\tilde{Q}$ 能够常驻在 L2 Cache，以避免其在内存和 Cache 之间的冗余数据搬运。更进一步，我们期望 KV 从内存到 Cache 的这一部分时延开销也能够被隐藏。

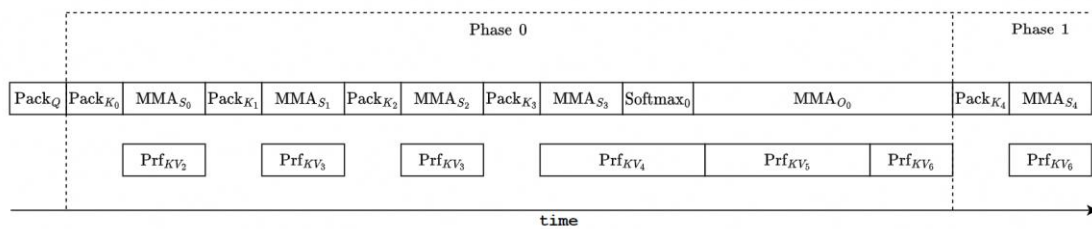
我们设计了基于预取的异步数据搬运：在 MMA 计算前总会利用预取 PRF 指令将对应 KV Cache 提前预取到 L2 Cache。为了更精细的控制预取行为，我们完全舍弃了硬件预取，采用纯软件的形式来预取所需的数据。

在具体实现中，考虑到 PRF 指令和 MOPA 指令会被发射到在不同的执行流水线，因此我们会在 MMA 阶段执行 MOPA 指令的同时穿插 PRF 指令预取后面的数据，隐藏 PRF 指令本身执行的时延。类似的，PRF 指令和 Softmax 过程中 SVE 相关的计算指令也能达到隐藏效果。

值得注意的是，虽然 KV 在内存中的排布并非完全连续（受 Page Block 存储形式影响），但在 Cacheline 大小内是连续的，因此具体实现中需要注意 Cacheline 对齐保证 Cache 命中率。

1.2.4 流水排布

如上所述，我们给出 $B_q = 32$ 的 Kernel 的各阶段流水排布，如下图：



可以发现，这样的排布能够处理任意足够长的 KV Cache：在 Phase i 的 MMA 和 Softmax 过程中会预取 Phase i 所需 KV Cache 的后半部分和 Phase $i + 1$ 所需 KV Cache 的前半部分。

类似地， $B_q = 64$ 的 Kernel 也设计了对应数据排布和流水排布，这里不再详细展开，具体可以参考对应的源码实现。

1.3 MLA 多线程方案

上面我们给出的 Kernel 最终会由单线程调用执行。为了充分发挥鲲鹏芯片众核架构的算力，我们设计了如下的 MLA 多线程方案。

1.3.1 并行度分析

回顾 MLA 计算的定义，可以发现在 N_{req} 和 m_q 这两个维度，上述 Kernel 并行计算没有任何依赖。

在 m_q 维度，对于给定的分块大小 B_q ，我们将 m_q 按照 B_q 为最小单位切块。

因此， m_q 维度的并行度为 $T_q = \left\lfloor \frac{m_q}{B_q} \right\rfloor$ 。

在 N_{req} 维度，我们同样可以多线程并行，并行度即 $T_{req} = N_{req}$ 。但是在 N_{req} 较小的情况下，并行度整体往往不能满足要求。同时，在不同 request 对应的序列长度差异较大的场景下会导致严重的负载不均衡。

因此，我们使用了在 KV Cache 的 s_{kv} 维度进一步按线程切分的多线程切分策略。

1.3.2 多线程策略

为了保证 SME 的计算效率，我们选择 $B_{seq} = 16$ 作为 KV Cache 切分的最小单位。关于具体的多线程策略，从以下 3 个方面评估线程端到端开销：

1. 各线程执行对应 KV 段的 MLA 计算的开销 W_0 。
2. 各线程在多段 KV 之间切换的开销 W_1 。
3. 切分 KV Cache 带来的线程间 Online-Softmax 合并开销 W_2 。

记 $S = \{s_i\}$ 为 MLA 多线程切分策略，其中 $s_i = \{kv_{i,0}, kv_{i,1}, \dots, kv_{i,n_i-1}\}$ 为线程 i 对应计算的 KV 段。端到端开销 $W(S)$ 可以表示为：

$$W(S) = \max_i (W_0(s_i) + W_1(n_i)) + W_2(S),$$

显然我们希望端到端开销最小，因此最优策略为：

$$S_{opt} = \underset{S}{\operatorname{argmin}} W(S).$$

1.4 参考性能数据

根据 DeepSeek V3 推理 Decode 实际场景进行测试：

计算瓶颈场景：

N_{MTP}	N_{req}	h_q	s_{kv}	计算效率	带宽效率
0	8	128	1024	51.45%	16.62%
0	8	128	8192	76.93%	20.69%
1	8	128	1024	50.18%	9.65%
1	8	128	8192	75.16%	10.40%

计算访存均衡场景：

N_{MTP}	N_{req}	h_q	s_{kv}	计算效率	带宽效率
0	32	32	1024	51.15%	56.61%
0	32	32	8192	64.13%	67.50%
1	32	16	1024	50.03%	55.37%
1	32	16	8192	62.24%	65.51%

1.5 总结及未来工作

未来进一步性能优化主要考虑从流水线和指令排布入手：

1. 短序列场景 MLA 的计算效率主要受暴露在流水线外的 Pack Q 和线程间 Online Softmax 影响。未来考虑将 Pack Q 融合进推理前置算子，MLA 侧提供对已经 Pack 的输入 Q 的支持。
2. 长序列场景 MLA 的计算效率则受限于未被 MMA 隐藏的 Pack K 和 Softmax 操作。未来进一步优化利用 SME 和 SVE 指令流水线的特性进行指令排布级的相互隐藏。